Mediapipe Gesture ID Sorting AprilTags

Mediapipe Gesture ID Sorting AprilTags

1. Course Content
2. Preparation
3. Case Demonstration
4. Core Source Code Analysis

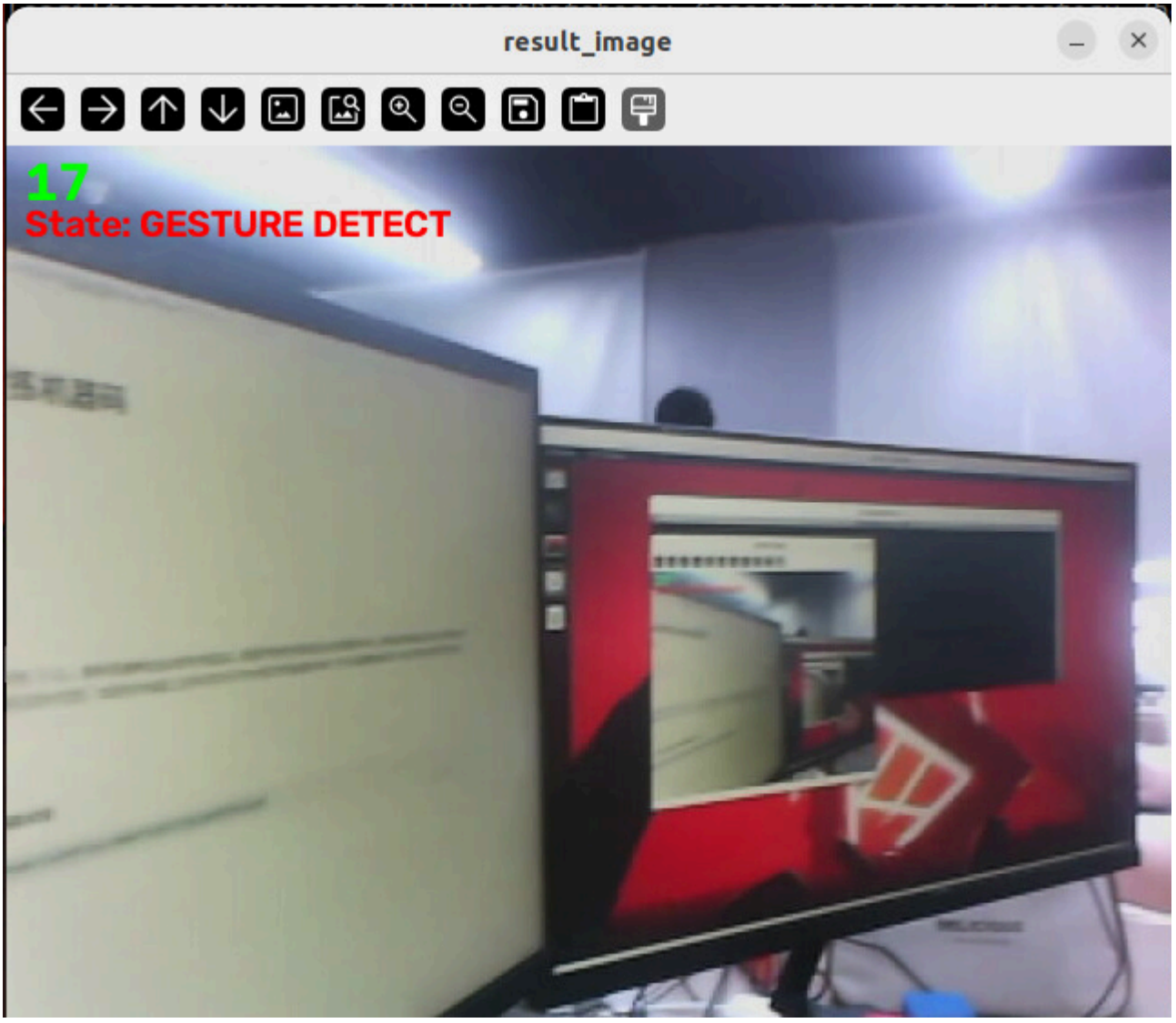
1. Course Content

The feature implements the program acquiring images through the camera and then recognizing gestures (1-4). The robotic arm will move to the sorting pose. It sorts the AprilTag wooden blocks on the tabletop according to the gesture: if an AprilTag wooden block with an ID corresponding to the gesture exists on the tabletop, the robotic arm lowers its gripper to grasp it and places it in a set position; if there is no AprilTag wooden block with an ID corresponding to the gesture on the tabletop, the robotic arm shakes its head and finally returns to the gesture recognition pose.

2. Preparation

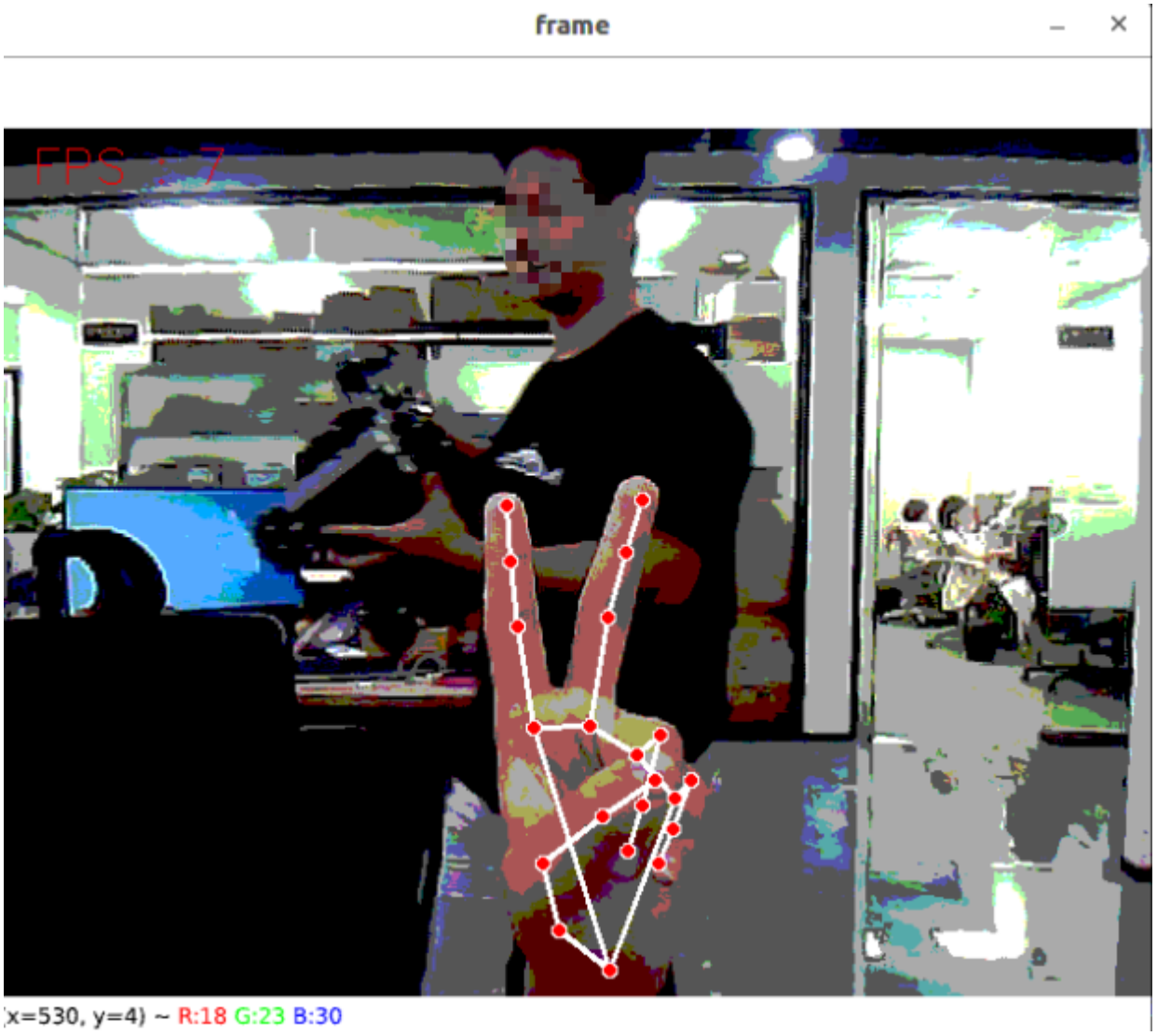
The wooden block used in this section is: **30x30x30mm AprilTag wooden block**

```
ros2 launch yahboomcar_bringup arm_demo.launch.py demo:=apriltag_gesture_sort
```

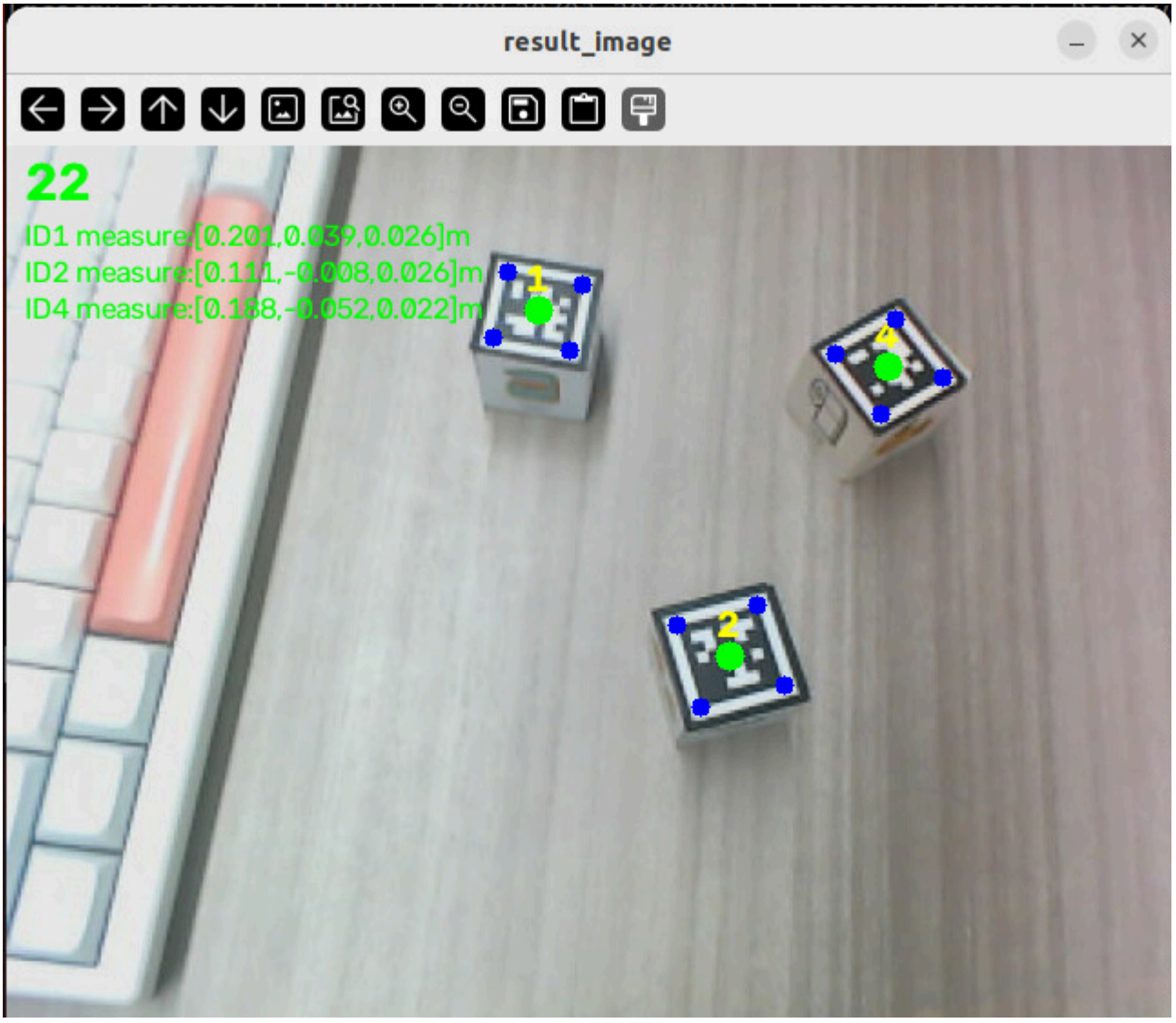


3. Case Demonstration

- Extend the hand in front of the camera (about 30~50cm from the lens), keep the palm open, and make a number gesture:
    - **Make gesture "1"**: the system recognizes Gesture=1, and the robotic arm locks onto and grasps AprilTag wooden block ID:1;
    - **Make gesture "2"**: the system recognizes Gesture=2, and the robotic arm locks onto and grasps AprilTag wooden block ID:2;
    - **Make gesture "3"**: the system recognizes Gesture=3, and the robotic arm locks onto and grasps AprilTag wooden block ID:3;
    - **Make gesture "4"**: the system recognizes Gesture=4, and the robotic arm locks onto and grasps AprilTag wooden block ID:4;



- After locking is confirmed, the buzzer emits a confirmation prompt tone, and the robotic arm automatically switches from the gesture interaction pose to the sorting pose, grasping the AprilTag with the corresponding number;



4. Core Source Code Analysis

- Program path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/common_demo/common_demo/arm_demo/apriltag_gesture_sort.py
```

pubSix_Arm_action function (controlling the robotic arm via Action):

This function controls the robotic arm to move to the specified joint angles through the ROS2 Action mechanism, supporting timeouts and asynchronous callbacks.

- `ArmPositionAction.Goal()`: constructs the robotic arm position control Action goal message
- `goal_msg.target_joints = [int(j) for j in joints[:6]]`: sets the target joint angles (taking the first 6 joints, converted to integers)
- `_arm_action_client.wait_for_server(timeout_sec=5.0)`: waits for the Action server to be ready; if it times out at 5 seconds, returns failure

- `threading.Event()`: creates a thread synchronization event, used to wait for the Action to complete
- `result_holder`: a dictionary structure that saves the Action execution result (success and message)
- `_goal_response_callback`: goal response callback; checks whether the goal is accepted, and registers a result callback if accepted
- `_result_callback`: result callback; saves the execution result and sets the event flag
- `send_goal_async(goal_msg)`: asynchronously sends the Action goal
- `done_event.wait(timeout=timeout)`: blocks and waits for the Action to complete; returns failure on timeout
- Returns `result_holder['success']`, indicating whether the execution succeeded

rgb_callback function (RGB image callback):

This function is the main callback, switching between the two modes of gesture detection and AprilTag sorting according to the state machine.

Pre-check:

- `self.camera_K`: camera intrinsic matrix; if not yet obtained, it waits and returns
- `self.bridge.imgmsg_to_cv2(msg, 'rgb8')`: converts the ROS Image message into an OpenCV RGB format image

Gesture detection state (STATE_GESTURE_DETECT):

- `cv2.cvtColor(result_image, cv2.COLOR_RGB2BGR)`: converts to BGR format for MediaPipe processing
- `self.hand_detector.findHands(result_image_bgr)`: calls the MediaPipe hand detector, returning the image with hand keypoints drawn, the keypoint list `lmList`, and the hand bounding box `bbox`
- `self._process_gesture_detect(lmList, result_image)`: processes the gesture detection result and recognizes the gesture number (1~4)
- In the gesture detection state, returns directly without performing AprilTag detection

AprilTag sorting state:

- `cv2.cvtColor(result_image, cv2.COLOR_RGB2GRAY)`: converts to grayscale for AprilTag detection
- `self.at_detector.detect(gray, estimate_tag_pose=True, ...)`: detects AprilTags, with pose estimation enabled and the camera parameters and tag size passed in
- `sorted(tags, key=lambda tag: tag.tag_id)`: sorts the detection results by tag_id
- `self._latest_tags = tags`: saves the latest detection result
- `draw_tags(result_image, tags, ...)`: draws the tag border (red) and center point (green) on the image
- `self._display_measure_pose(tags, result_image)`: displays the pose of each tag in the measure coordinate frame

Sorting logic:

- If no tag is detected or grasping is in progress, returns directly
- `self.compute_flag`: sorting enable flag, set to True after gesture confirmation
- `self.sorting_start_time`: sorting start time, used for timeout checking

- `self.sorting_timeout`: sorting timeout (seconds); after timeout, calls `_sorting_timeout()` to fall back to the detection pose
- `self._find_target_tag(tags, self.gesture_target_id)`: finds the AprilTag matching the gesture target ID in the detection results
- `self.selected_tag_index`: if the target tag is found, calls `_process_grasp()` to execute the grasping flow
- `self._display_fps(result_image)`: displays the frame rate

```
    def pubSix_Arm_action(self, joints, timeout=30.0):
        goal_msg = ArmPositionAction.Goal()
        goal_msg.target_joints = [int(j) for j in joints[:6]]

        if not self._arm_action_client.wait_for_server(timeout_sec=5.0):
            self.get_logger().error("Arm action server not available")
            return False

        done_event = threading.Event()
        result_holder = {'success': False, 'message': ''}

        def _goal_response_callback(future):
            goal_handle = future.result()
            if not goal_handle.accepted:
                self.get_logger().error("Arm action goal rejected")
                done_event.set()
                return
            get_result_future = goal_handle.get_result_async()
            get_result_future.add_done_callback(_result_callback)

        def _result_callback(future):
            res = future.result().result
            result_holder['success'] = res.success
            result_holder['message'] = res.message
            done_event.set()

        send_goal_future = self._arm_action_client.send_goal_async(goal_msg)
        send_goal_future.add_done_callback(_goal_response_callback)

        if not done_event.wait(timeout=timeout):
            self.get_logger().error("Arm action timed out")
            return False

        if result_holder['success']:
            self.get_logger().info(f"Arm action completed: {result_holder['message']}")
        return result_holder['success']

    def rgb_callback(self, msg):

        if self.camera_K is None:
            self.get_logger().warn("Waiting for camera intrinsics...")
            return
```

```
        result_image = self.bridge.imgmsg_to_cv2(msg, 'rgb8')

        # ========== Gesture detection state ==========
        if self.state == self.STATE_GESTURE_DETECT:
            # MediaPipe gesture detection (BGR format) -- only runs in the gesture detection
state
            result_image_bgr = cv2.cvtColor(result_image, cv2.COLOR_RGB2BGR)
            result_image_bgr, lmList, bbox = self.hand_detector.findHands(result_image_bgr)
            result_image = cv2.cvtColor(result_image_bgr, cv2.COLOR_BGR2RGB)
            self._process_gesture_detect(lmList, result_image)
            self._display_fps(result_image)
            return

        gray = cv2.cvtColor(result_image, cv2.COLOR_RGB2GRAY)
        tags = self.at_detector.detect(
            gray,
            estimate_tag_pose=True,
            camera_params=self.camera_params,
            tag_size=self.tag_size
        )
        tags = sorted(tags, key=lambda tag: tag.tag_id)

        self._latest_tags = tags

        draw_tags(result_image, tags, corners_color=(0, 0, 255), center_color=(0, 255, 0))

        self._display_measure_pose(tags, result_image)


        if len(tags) == 0:
            self._display_fps(result_image)
            return

        if self._grasp_in_progress:
            self._display_fps(result_image)
            return

        # Timeout check: must come after the _grasp_in_progress check to avoid false
triggers during grasping
        if self.compute_flag and self.sorting_start_time is not None:
            if time.time() - self.sorting_start_time > self.sorting_timeout:
                self.get_logger().warn(
                    f"Timeout: target tag ID {self.gesture_target_id} not found within "
                    f"{self.sorting_timeout:.0f}s, returning to detection pose"
                )
                self._sorting_timeout()
                return

        if self.compute_flag:
            self._find_target_tag(tags, self.gesture_target_id)

        if self.selected_tag_index is not None:
            self._process_grasp(tags, result_image)
```

```
        # Display the results
        self._display_fps(result_image)
```

- Parameter file

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/param/common.yaml
```

Parameter Description:

- `tag_size: 0.02`: the physical side length of the AprilTag (meters), used for pose estimation
- `apriltag_grasp_distance: [0.11, 0.0]`: the expected grasping distance during chassis alignment `[x, y]` (meters), in the measure coordinate frame
- `observation_pose: [90, 107, 0, 0, 90, 12]`: the 6 joint angles (degrees) of the observation pose, at which the robotic arm has the best field of view
- `grasp_offset: [0.00, 0.0, 0.015, 85.0]`: grasping offset `[x, y, z, pitch]`, compensating for the error reaching the target point caused by servo backlash and link rigidity degradation; pitch=85° means the gripper grasps almost vertically downward
- `gripping_angle: 140`: the angle (degrees) when the gripper is closed; the larger the value, the tighter the gripper
- `detection_pose: [90, 150, 12, 20, 90, 30]`: the robotic arm pose during gesture detection, at which the camera's field of view faces the user's hand region
- `sorting_pose: [90, 107, 0, 0, 90, 30]`: the robotic arm pose during sorting and grasping; after switching to this pose, AprilTag detection and grasping are performed
- `gesture_stable_threshold: 15`: gesture stable frame count threshold; the same gesture must be recognized for 15 consecutive frames before confirmation, preventing false triggering
- `sorting_timeout: 3.0`: sorting timeout (seconds); if the AprilTag with the target ID is not found within the specified time, it falls back to the detection pose to re-recognize the gesture

```
apriltag_gesture_sort:
  ros__parameters:
    tag_size: 0.02                          # AprilTag side length (meters)
    apriltag_grasp_distance: [0.11, 0.0]    # measure coordinate frame x,y offset (meters)
    observation_pose: [90, 107, 0, 0, 90, 12]
    grasp_offset: [0.00,0.0,0.015,85.0]     # x,y,z axis offset distance and included angle
    gripping_angle: 140                     # gripper grasping angle, in degrees, integer
    detection_pose: [90, 150, 12, 20, 90, 30]  # robotic arm pose during gesture detection
    sorting_pose: [90, 107, 0, 0, 90, 30]   # robotic arm pose during sorting and grasping
    gesture_stable_threshold: 15            # gesture stable frame count threshold
    sorting_timeout: 3.0                    # sorting timeout (seconds); falls back to the
detection pose after timeout
```